

Adding Self-Reflection to the Workflow

We now have a deep research agent! It comprehensively researches a topic for us before providing a detailed answer. But we can do better!

🧘 LLMs are capable of self-reflection: they can read their own work, critique it, and provide feedback, allowing them to take a second try when they fall short.

Let's add reflection to our deep research agent! This will involve several changes:

- In `research` we'll store the research into the context, since we might need to use it multiple times
- We'll tell `write` that it can be triggered by a `RewriteEvent` in addition to a `WriteEvent`
- If it's a `RewriteEvent` we'll add the review as feedback to the prompt
- `review` will be changed to optionally emit a `RewriteEvent`
- We'll get the LLM to decide if the review returned by the agent is a "bad" or "good" review

! Note: Run the hidden cells below before running the rest of the code. !

```
!pip install llama-index -q -q
```

```
!pip install tavily-python -q -q
```

```
from tavily import AsyncTavilyClient
from llama_index.core.agent.workflow import AgentWorkflow
from llama_index.llms.openai import OpenAI
import os
from openai import OpenAI as OpenAIClient

raw_client = OpenAIClient()

API_KEY = raw_client.api_key
API_BASE = raw_client.base_url

tavily_api_key = os.environ["TAVILY_API_KEY"]

async def search_web(query: str) -> str:
    """Useful for using the web to answer questions."""
    client = AsyncTavilyClient(api_key=tavily_api_key)
    return str(await client.search(query))

llm = OpenAI(model="gpt-4o-mini", api_key=API_KEY, api_base=API_BASE)

workflow = AgentWorkflow.from_tools_or_functions(
    [search_web],
    llm=llm,
    system_prompt="You are a helpful assistant that answers questions. If you don't know the answer, you can search the web for information.",
)
```

```

from llama_index.core.agent.workflow import FunctionAgent
from llama_index.core.workflow import (
    Event,
    StartEvent,
    StopEvent,
    Context,
    Workflow,
    step
)

class FeedbackEvent(Event):
    feedback: str

class ReviewEvent(Event):
    report: str

class GenerateEvent(Event):
    research_topic: str

class QuestionEvent(Event):
    question: str

class AnswerEvent(Event):
    question: str
    answer: str

class ProgressEvent(Event):
    msg: str

question_agent = FunctionAgent(
    tools=[],
    llm=llm,
    verbose=False,
    system_prompt="""You are part of a deep research system.
    Given a research topic, you should come up with a bunch of questions
    that a separate agent will answer in order to write a comprehensive
    report on that topic. To make it easy to answer the questions separately,
    you should provide the questions one per line. Don't include markdown
    or any preamble in your response, just a list of questions."""
)

answer_agent = FunctionAgent(
    tools=[search_web],
    llm=llm,
    verbose=False,
    system_prompt="""You are part of a deep research system.
    Given a specific question, your job is to come up with a deep answer
    to that question, which will be combined with other answers on the topic

```

```
        into a comprehensive report. You can search the web to get information
        on the topic, as many times as you need."""
    )
    report_agent = FunctionAgent(
        tools=[],
        llm=llm,
        verbose=False,
        system_prompt="""You are part of a deep research system.
        Given a set of answers to a set of questions, your job is to combine
        them all into a comprehensive report on the topic."""
    )
```

```

review_agent = FunctionAgent(
    tools=[],
    llm=llm,
    verbose=False,
    system_prompt="""You are part of a deep research system.
    Your job is to review a report that's been written and suggest
    questions that could have been asked to produce a more comprehensive
    report than the current version, or to decide that the current
    report is comprehensive enough."""
)

class DeepResearchWithReflectionWorkflow(Workflow):

    @step
    async def setup(self, ctx: Context, ev: StartEvent) -> GenerateEvent:
        self.question_agent = ev.question_agent
        self.answer_agent = ev.answer_agent
        self.report_agent = ev.report_agent
        self.review_agent = ev.review_agent
        self.review_cycles = 0

        ctx.write_event_to_stream(ProgressEvent(msg="Starting research"))

        return GenerateEvent(research_topic=ev.research_topic)

    @step
    async def generate_questions(self, ctx: Context, ev: GenerateEvent |
FeedbackEvent) -> QuestionEvent:

        await ctx.set("research_topic", ev.research_topic)
        ctx.write_event_to_stream(ProgressEvent(msg=f"Research topic is
{ev.research_topic}"))

        prompt = f"""Generate some questions on the topic <topic>{ev.research_topic}
</topic>."""

        if isinstance(ev, FeedbackEvent):
            ctx.write_event_to_stream(ProgressEvent(msg=f"Got feedback:
{ev.feedback}"))
            prompt += f"""You have previously researched this topic and
            got the following feedback, consisting of additional questions
            you might want to ask: <feedback>{ev.feedback}</feedback>.
            Keep this in mind when formulating your questions."""

        result = await self.question_agent.run(user_msg=prompt)

        # Some basic string manipulation to get separate questions

```

```

lines = str(result).split("\n")
questions = [line.strip() for line in lines if line.strip() != ""]

# Record how many answers we're going to need to wait for
await ctx.set("total_questions", len(questions))

# Fire off multiple Answer Agents
for question in questions:
    ctx.send_event(QuestionEvent(question=question))

@step
async def answer_question(self, ctx: Context, ev: QuestionEvent) -> AnswerEvent:

    result = await self.answer_agent.run(user_msg=f"""Research the answer to
this
    question: <question>{ev.question}</question>. You can use web
    search to help you find information on the topic, as many times
    as you need. Return just the answer without preamble or markdown.""")

    ctx.write_event_to_stream(ProgressEvent(msg=f"""Received question
{ev.question}
    Came up with answer: {str(result)}"""))

    return AnswerEvent(question=ev.question, answer=str(result))

@step
async def write_report(self, ctx: Context, ev: AnswerEvent) -> ReviewEvent:

    # CODE: store the answers in a variable
    research = ctx.collect_events(ev, [AnswerEvent] * await
ctx.get("total_questions"))
    # If we haven't received all the answers yet, this will be None
    if research is None:
        ctx.write_event_to_stream(ProgressEvent(msg="Collecting answers..."))
        return None

    ctx.write_event_to_stream(ProgressEvent(msg="Generating report..."))

    # Aggregate the questions and answers
    all_answers = ""
    for q_and_a in research:
        all_answers += f"Question: {q_and_a.question}\nAnswer:
{q_and_a.answer}\n\n"

    # Prompt the report
    result = await self.report_agent.run(user_msg=f"""You are part of a deep
research system.
    You have been given a complex topic on which to write a report:
    <topic>{await ctx.get("research_topic")}</topic>.""")

```

Other agents have already come up with a list of questions about the topic and answers to those questions. Your job is to write a clear, thorough report that combines all the information from those answers.

Here are the questions and answers:

```
<questions_and_answers>{all_answers}</questions_and_answers>""")
```

```
return ReviewEvent(report=str(result))
```

```
@step
```

```
async def review(self, ctx: Context, ev: ReviewEvent) -> StopEvent |
```

```
FeedbackEvent:
```

```
# CODE: call the review agent at this step
```

```
result = await self.review_agent.run(user_msg=f"""\nYou are part of a deep\nresearch system.
```

```
    You have just written a report about the topic {await\nctx.get("research_topic")}).
```

```
    Here is the report: <report>{ev.report}</report>
```

```
    Decide whether this report is sufficiently comprehensive.
```

```
    If it is, respond with just the string "ACCEPTABLE" and nothing else.
```

```
    If it needs more research, suggest some additional questions that could\n    have been asked.""")
```

```
self.review_cycles += 1
```

```
# Either it's okay or we've already gone through 3 cycles
```

```
if str(result) == "ACCEPTABLE" or self.review_cycles >= 3:
```

```
    return StopEvent(result=ev.report)
```

```
else:
```

```
    ctx.write_event_to_stream(ProgressEvent(msg="Sending feedback"))
```

```
    return FeedbackEvent(
```

```
        research_topic=await ctx.get("research_topic"),
```

```
        feedback=str(result)
```

```
)
```

 **Note:** As in the previous exercise, we've cached the report for the topic "History of San Francisco" to keep this solution fast so that you can see the outcome instantly.

If you'd like to run the full agentic workflow with reflection, you can run the code below. This will use the actual agents to generate the report from scratch.

We recommend this for your own projects when you're ready to explore the complete workflow, as it may involve more extensive processing times.

```
workflow = DeepResearchWithReflectionWorkflow(timeout=300)
handler = workflow.run(
    research_topic="History of San Francisco",
    question_agent=question_agent,
    answer_agent=answer_agent,
    report_agent=report_agent,
    review_agent=review_agent
)

async for ev in handler.stream_events():
    if isinstance(ev, ProgressEvent):
        print(ev.msg)

final_result = await handler
print("==== The report ====")
print(final_result)
```

 Copy

! Note: Run the hidden cell below before running the rest of the code. !

```
import json
import os
import asyncio

CACHE_PATH = "sf_history_reflection_cache.json"
CACHE_KEY = "History of San Francisco"

# Load or initialize the cache
if os.path.exists(CACHE_PATH):
    with open(CACHE_PATH, "r") as f:
        full_cache = json.load(f)
else:
    full_cache = {}

if CACHE_KEY in full_cache:
    cached_result = full_cache
```

```
async def replay_cached():
    messages = [
        "Starting research...",
        f"Research topic is: {CACHE_KEY}",
        "Collecting answers...",
        "Generating report...",
        "Reflecting on report...",
        "Sending feedback...",
        "Generating report...",
        "Reflecting on report...",
        "Sending feedback...",
        "Generating report...",
        "Reflecting on report...",
    ]
    for msg in messages:
        await asyncio.sleep(0.4)
        print(msg)

    print("==== The report ====")
    print(cached_result)

await replay_cached()
```

```
Starting research...
Research topic is: History of San Francisco
Collecting answers...
Generating report...
Reflecting on report...
Sending feedback...
Generating report...
Reflecting on report...
Sending feedback...
Generating report...
Reflecting on report...
==== The report ====
# Comprehensive Report on the History of San Francisco
```

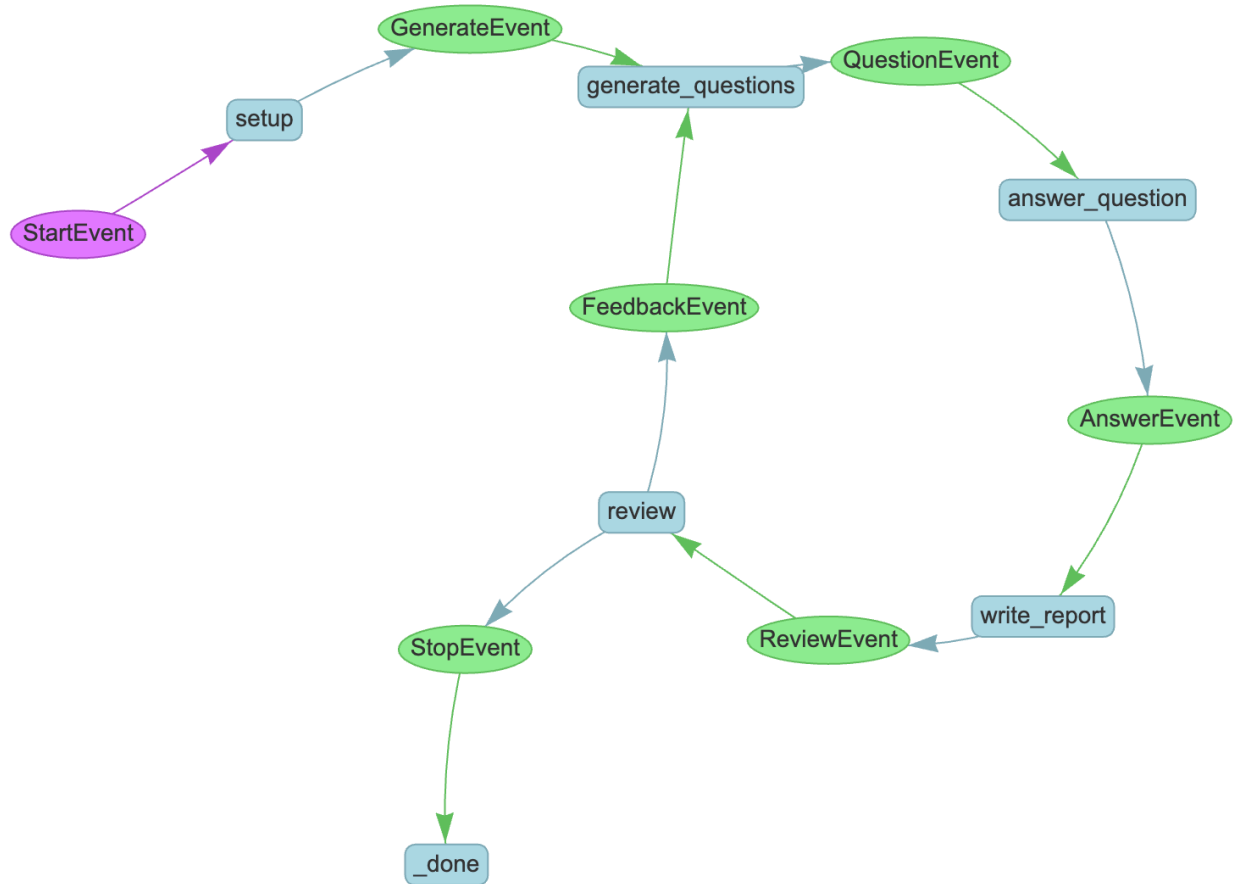
San Francisco, a city known for its iconic landmarks and vibrant culture, has a rich and complex history shaped by various events, demographics, and social movements. This report delves into the key historical milestones, cultural influences, and socio-economic developments that have defined San Francisco from its early days to the present.

Founding and Early Development

The history of San Francisco begins with the Indigenous Ohlone people, who inhabited

the Bay Area for thousands of years before European contact. The arrival of Spanish explorers in 1769 marked the beginning of European settlement, culminating in the establishment of the Presidio of San Francisco and Mission San Francisco de Asís in 1776. Following Mexico's independence from Spain in 1821, the area remained sparsely populated until the discovery of gold at Sutter's Mill in 1848, which triggered the

In this run, we did three rounds of review! You can see the full looping workflow in the visualizer:



One final congratulations!

You've come a long way in this notebook — from a single agent flow, to a multi-agent `AgentWorkflow` all the way through to full `Workflows`, with fine-grained flow control and self-reflection. You're ready to build your multi-agent system to tackle even more projects!

